

Pizza Ordering Service

Dönem Projesi Final Raporu

Bulut Mimarilerinde Test Mühendisliği | MTH2526-B25
Marmara Üniversitesi — Bilgisayar Mühendisliği Bölümü

Yasir ARSLAN — 170423528
Büşra Ecem ÖZBEK — 170423966
Haziran 2026

1. Giriş

Bu çalışmada, Bulut Mimarilerinde Test Mühendisliği dersi kapsamında **Pizza Ordering Service** adlı bir REST API mikroservisi geliştirilmiş ve endüstri standardında bir test ve dağıtım altyapısı kurulmuştur. Konu olarak pizza sipariş servisi seçilmesinin temel nedeni, domain'in yalın ve anlaşılır olması, buna karşın sipariş oluşturma, fiyatlandırma, durum geçişleri ve kupon yönetimi gibi test edilebilir iş kuralları barındırmasıdır.

Projenin amacı karmaşık bir uygulama yazmak değil; basit bir uygulama üzerinden unit test, integration test, E2E test, CI/CD pipeline, container, Kubernetes, monitoring ve performans testi katmanlarının tamamını uygulamalı olarak göstermektir.

2. Mimari

Uygulama, Docker Compose ile yönetilen çok servisli bir mikroservis mimarisiyle tasarlanmıştır. 7 servis tek komutla ayağa kalkar: app, postgres, localstack, prometheus, grafana, otel-collector, jaeger.

2.1 Bileşenler

- FastAPI (Python 3.12):** 7 endpoint sunar — /health, /menu, /orders (CRUD), durum güncelleme, kupon uygulama.
- SQLAlchemy + PostgreSQL 16:** ORM tabanlı veri katmanı. Lokalde SQLite, Docker/prod'da PostgreSQL kullanılır.
- LocalStack S3:** Sipariş özeti her oluşturmada orders/{id}.json anahtarıyla S3'e arşivlenir.
- Prometheus + Grafana:** /metrics toplanır; Request Throughput, Error Rate, Latency P95 panelleri.
- OpenTelemetry + Jaeger:** FastAPI, SQLAlchemy ve AWS çağrıları otomatik trace edilir.
- KEDA:** Prometheus metriğine göre 1–5 replica arasında otomatik ölçekleme.
- ArgoCD + Helm:** GitOps ile main branch'ten otomatik K8s deploy; Helm chart ile merkezi yönetim.

2.2 API Endpoint Tablosu

Method	Endpoint	Açıklama
GET	/health	Sağlık kontrolü (CI smoke + K8s probe)
GET	/menu	Aktif menü öğelerini listele
POST	/orders	Yeni sipariş oluştur (201)
GET	/orders	Tüm siparişleri listele
GET	/orders/{id}	Tek sipariş detayı (404 yoksa)
PATCH	/orders/{id}/status	Durum güncelle (pending→preparing→ready→delivered)
POST	/orders/{id}/apply-coupon	Kupon uygula (PIZZA10 / CHEESE20)

3. Test Stratejisi

Test piramidi yaklaşımı benimsenmiştir. Toplamda **18 test, %86 code coverage** (şart: %70).

3.1 Unit Testler — 11 adet (tests/unit/)

SQLite in-memory DB kullanılır, 0.30 sn'de tamamlanır.

- test_health.py: /health → 200 + {"status":"ok"}
- test_orders_api.py (8 test): Sipariş oluşturma, geçersiz extra (pineapple → 422), çift kupon → 400, geçersiz kupon kodu → 400, geçersiz durum geçişi (pending→delivered → 400), sıralı durum ilerlemesi, S3 arşiv tetiklenme.
- test_order_archive.py: Factory Boy sahte sipariş → S3 JSON formatı doğrulaması.
- test_web_ui.py: Ana sayfa HTML içeriği kontrolü.

3.2 Integration Testler — 3 adet (tests/integration/)

Testcontainers ile gerçek PostgreSQL 16 ve LocalStack container'ları başlatılır.

- Sipariş gerçek DB'ye yazılıyor mu? Fiyat hesabı doğru mu?
- PIZZA10 kuponu sonrası DB'deki total_price güncelleniyor mu?
- Sipariş sonrası S3'te orders/{id}.json oluşuyor mu?

3.3 E2E Testler — 4 adet (tests/e2e/)

Playwright Chromium ile gerçek tarayıcı, ayrı thread'de uvicorn sunucusu.

- Sipariş oluşturma formu → yanıt doğrulama.
- Sipariş listeleme ve detay görüntüleme.
- Geçersiz extra → hata mesajı sayfada görünüyor mu?

3.4 Test Verisi Üretimi

Factory Boy + Faker ile MenuItemFactory, OrderItemFactory, OrderFactory sınıfları yazılmıştır. Her çağrıda gerçekçi, rastgele ama geçerli veri üretilir.

4. Pipeline ve Deploy

4.1 GitHub Actions CI/CD

#	Adım
1-4	Checkout → Python 3.11 → Node.js → kubectl → kind cluster
5-6	pip install -e .[dev] + newman + Playwright Chromium
7	Lint: ruff check .
8	pytest --cov=src --cov-fail-under=70
9-10	docker build + kind'a image yükle
11-12	kubectl apply (configmap, secret, service, deployment) + rollout status
13	Smoke test: port-forward + curl /health + Newman

4.2 Kubernetes Manifestleri

- **deployment.yaml**: 1 replica, /health probe, CPU 500m / RAM 256Mi limit.
- **service.yaml**: ClusterIP, port 80 → container 8000.
- **configmap.yaml**: Hassas olmayan env değişkenleri (DATABASE_URL vb.).
- **secret.yaml**: AWS anahtarları (hassas değişkenler).

4.3 Helm Chart ve ArgoCD

- charts/pizza-ordering-service/ — values.yaml ile image, replica, port merkezi yönetim.
- ArgoCD: main branch'e merge → otomatik cluster deploy. prune + selfHeal aktif.

5. Performans ve Gözlemlenebilirlik

5.1 Performans Testi

k6 (perf/load-test.js) ve Locust (perf/locustfile.py) ile /menu, POST /orders ve GET /orders endpoint'lerine eş zamanlı yük uygulanmıştır. p95 latency ölçümü sonuçları perf/report.md dosyasında raporlanmıştır.

5.2 Grafana Dashboard (3 Panel)

- **Request Throughput (req/s):** Trafik yoğunluğunu gösterir.
- **Error Rate:** HTTP 5xx hata oranı.
- **Latency P95:** İsteklerin %95'inin yanıt süresi.

5.3 OpenTelemetry Distributed Tracing

FastAPI, SQLAlchemy ve Botocore otomatik enstrümantasyonla izlenmektedir. Trace'ler Jaeger UI'da (localhost:16686) görüntülenebilir. Bir isteğin API'den DB'ye, oradan S3'e kadar tüm süreci tek ekranda takip edilebilir.

6. Sonuç ve Öğrendiklerimiz

6.1 Sayısal Özet

Metrik	Değer
Toplam test	18 (11 unit + 3 integration + 4 E2E)
Code coverage	%86 — şart %70, hedef aşıldı
API endpoint	7
Docker servis	7
CI/CD adım	13
Bonus katman	4 × +5 = +20 puan (Helm, KEDA, ArgoCD, OpenTelemetry)

6.2 Öğrendiklerimiz

- **Test katmanları:** Her katmanın farklı amacı olduğu pratik olarak gözlemlendi — unit hız, integration gerçeklik, E2E kullanıcı senaryosu.
- **Container teknolojileri:** Docker Compose ile tekrar üretilebilir çok servisli ortam yönetimi deneyimi kazanıldı.
- **Gözlemlenebilirlik:** Prometheus + Grafana + OpenTelemetry üçlüsünün birlikte nasıl çalıştığı kavrandı.

6.3 Zorluklar

- Docker kurulumunda WSL 2 gerekliliği beklenmedik bir engel oluşturdu.
- Testcontainers'ın Windows ortamında Docker Desktop yapılandırması gerekti.
- Kubernetes manifest yazımı ve servisler arası bağımlılıklar başlangıçta karmaşık geldi.

7. İş Paylaşımı

Dosya / Modül	Sorumlu	Yardımcı
src/main.py, src/api/routes.py	Yasir ARSLAN	B. Ecem ÖZBEK
src/services/orders.py, pricing.py	Yasir ARSLAN	B. Ecem ÖZBEK
src/integrations/order_archive.py, tracing.py	Yasir ARSLAN	—
src/models/, src/schemas/, src/db/	B. Ecem ÖZBEK	Yasir ARSLAN
tests/unit/ (tüm unit testler)	Yasir ARSLAN	B. Ecem ÖZBEK
tests/unit/test_web_ui.py, tests/integration/	B. Ecem ÖZBEK	Yasir ARSLAN
tests/conftest.py, factories.py	B. Ecem ÖZBEK	Yasir ARSLAN
tests/e2e/test_ui_flow.py	Yasir ARSLAN	B. Ecem ÖZBEK
Dockerfile, docker-compose.yml	Yasir ARSLAN	B. Ecem ÖZBEK
k8s/deployment.yaml, service.yaml, scaledobject.yaml, argocd-application.yaml	Yasir ARSLAN	—
k8s/configmap.yaml, secret.yaml	B. Ecem ÖZBEK	—
.github/workflows/ci.yml	Yasir ARSLAN	B. Ecem ÖZBEK
monitoring/prometheus.yml, grafana-dashboard.json	B. Ecem ÖZBEK	Yasir ARSLAN
monitoring/otel-collector-config.yaml	Yasir ARSLAN	—
charts/pizza-ordering-service/ (Helm)	Yasir ARSLAN	B. Ecem ÖZBEK
perf/load-test.js, perf/locustfile.py	B. Ecem ÖZBEK	—
postman/collection.json	Yasir ARSLAN	—
docs/architecture.md, architecture.png	B. Ecem ÖZBEK	Yasir ARSLAN
pyproject.toml, README.md	Yasir ARSLAN	B. Ecem ÖZBEK

Sunum Sorumluluğu

- 0–7 dk: Problem + Mimari + Test Stratejisi — Yasir ARSLAN
- 7–14 dk: Canlı Demo (docker compose, API, Grafana, testler) — Büşra Ecem ÖZBEK
- 14–17 dk: Sayılar + Öğrendiklerimiz — Yasir ARSLAN
- 17–20 dk: Q&A; — İkisi birlikte

8. Kaynaklar

- [1] FastAPI Resmi Dokümantasyonu — <https://fastapi.tiangolo.com>
- [2] SQLAlchemy Dokümantasyonu — <https://docs.sqlalchemy.org>
- [3] Docker Resmi Dokümantasyonu — <https://docs.docker.com>
- [4] Testcontainers Python — <https://testcontainers-python.readthedocs.io>
- [5] Playwright Python — <https://playwright.dev/python>
- [6] Prometheus + Grafana Dokümantasyonu — <https://prometheus.io/docs>
- [7] OpenTelemetry Python — <https://opentelemetry.io/docs/instrumentation/python>
- [8] KEDA Dokümantasyonu — <https://keda.sh/docs>
- [9] ArgoCD Dokümantasyonu — <https://argo-cd.readthedocs.io>
- [10] Kubernetes Resmi Dokümantasyonu — <https://kubernetes.io/docs>
- [11] k6 Yük Testi — <https://k6.io/docs> | Locust — <https://docs.locust.io>
- [12] Docker Eğitim Videoları — YouTube